

Politechnika Wrocławska

Bazy danych - Projektowanie

Mateusz Stochel 272524
Szymon Senderowski 272603

Prowadzący: mgr inż. Michał Jaroszczuk
środa 18:55 - 20:35 TN

30 października 2026

Spis treści

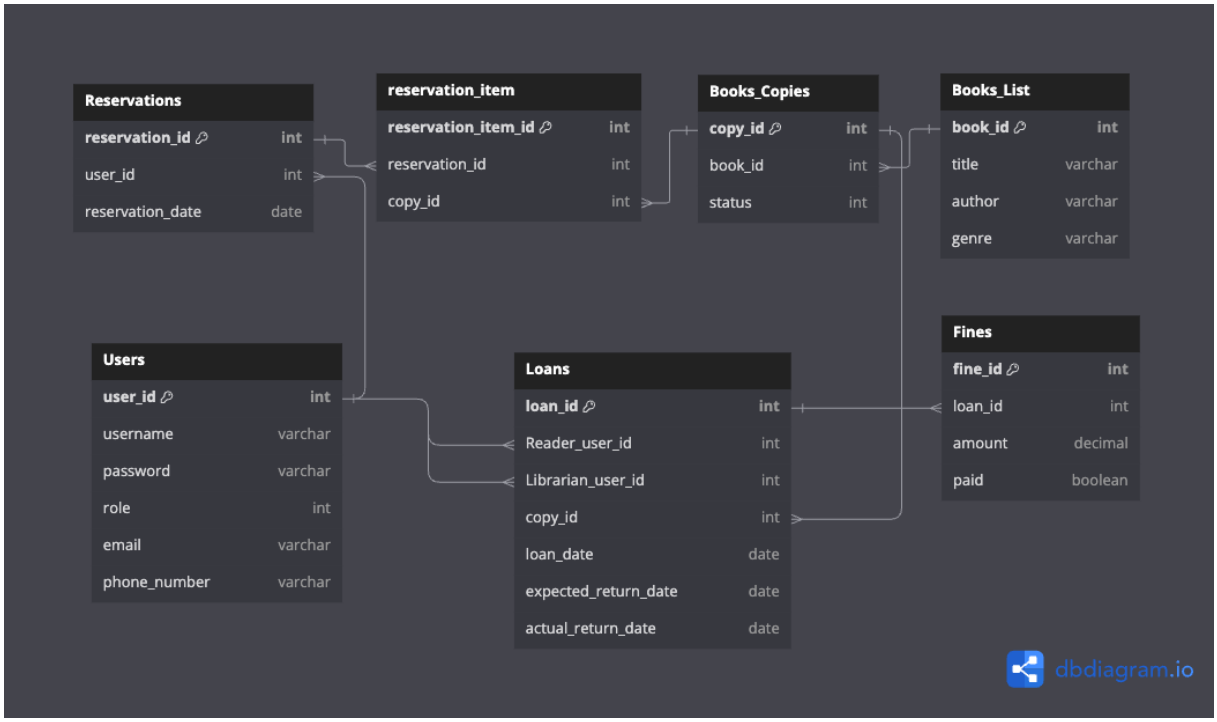
1 Baza danych **3**

2 Aplikacja **4**
2.1 Opis 4
2.2 Funkcjonalności 4

3 Opis endpointów **4**

Dokumentacja do projektu, którym jest aplikacja do obsługi biblioteki. Link do repozytorium na github: `https://github.com/Zimi03/Library`

1 Baza danych



Rysunek 1: Projekt bazy danych

Baza przechowuje wszystkie dane potrzebne do obsługi biblioteki. Role użytkowników są przechowywane w zmiennej całkowitej:

- 0 - Czytelnik
- 1 - Bibliotekarz
- 2 - Administrator

Zmienna dotycząca statusu książki również jest opisana zmienną całkowitą:

- 0 - wycofana
- 1 - dostępna
- 2 - wypożyczona
- 3 - zarezerwowana

W tabeli Loans:

- Pole "loan_date"jest datą dnia, w którym książka została wypożyczona
- Pole "expected_return_date"jest to dzień po którym czytelnik zwróci książkę bez płacenia kary - 30 dni od wypożyczenia
- Pole "actual_return_date jest to dzień w którym użytkownik rzeczywiście oddał książkę

2 Aplikacja

2.1 Opis

Aplikacja służy do obsługi biblioteki. Ma ona ułatwić proces wypożyczania książek, rezerwacji czy zarządzania użytkownikami.

2.2 Funkcjonalności

1. Zarządzanie użytkownikami - aplikacja pozwala na rejestracje i przechowywanie danych na temat użytkowników. Użytkownicy mają różne uprawnienia (Czytelnik, Bibliotekarz, Admin). O uprawnieniach poszczególnych typów użytkowników będzie w opisie endpointów.

2. Dodawanie nowych książek - Bibliotekarz może dodać dodać nowy egzemplarz już dostępnej książki (wtedy dodawany jest rekord do tabeli Books_Copies) bądź dodać zupełnie nową książkę dotąd niedostępną w bazie (Wtedy dodawany jest rekord w tabeli Books_List i Books_Copies).

3. Wyszukiwanie książek - każdy użytkownik może sprawdzić jakie książki są w bazie biblioteki i czy są one obecnie dostępne do wypożyczenia.

4. Usuwanie książek z bazy - Jest to funkcja dostępna dla bibliotekarza. Nie usuwa ona rekordu z bazy, tylko ustawia status książki na 0.

5. Rezerwacja książki - każdy użytkownik może zarezerwować kilka książek na raz na okres jednego tygodnia. Rezerwacja zmienia status wybranych książek na 3 i dodaje odpowiednie wpisy do tabel - reservation_item i Reservations.

6. Wypożyczenie książki - Tę funkcję może używać tylko zalogowany bibliotekarz. Dany użytkownik może wypożyczyć jedną książkę na raz. Dodaje ona odpowiedni wpis do tabeli Loans i ustawia status wypożyczonej książki na 2. Daty opisane zostały w opisie tabeli Loans 1. Użytkownik nie może wypożyczyć książki dopóki nie zapłaci kary, którą mógł dostać przy zwrocie poprzedniej książki.

7. Zwrot książki - Zanotować zwrot książki również może tylko zalogowany bibliotekarz. Ta funkcja aktualizuje: status zwróconej książki na 0, pole "actual_return_date"w tabeli na dzień w którym oddał książkę. Jeśli data oddania książki jest późniejsza niż data oczekiwanego zwrotu naliczana jest kara w wysokości 0.30 zł za dzień spóźnienia. Wtedy dodawany jest również wpis do tabeli Fines o zadanym loan_id, wysości kary równej tak jak opisano wyżej i niezapłaconym statusie.

8. Sprawdzić obecne kary użytkownika - tę funkcję może wykonać każdy użytkownik aby sprawdzić jaką karę ma do zapłacenia

9. Płatność kary - z tej funkcji może skorzystać tylko bibliotekarz, aby inny użytkownik mógł zapłacić karę. Zmienia pole "paid"w tabeli Fines przy odpowiedniej każe.

10. Sprawdzenie wszystkich wypożyczeń użytkownika - Ta funkcja jest dostępna dla każdego użytkownika, pozwala ona sprawdzić wszystkie dotychczasowe wypożyczenia danego użytkownika

11. Sprawdzenie wsyzstkich rezerwacji użytkownika - Ta funkcja również jest dostępna dla wyszystkich użytkowników, pozwala ona sprawdzić wszystkie dotychczasowe rezerwacje danego użytkownika

3 Opis endpointów

1. POST addUser - dodaje nowego czytelnika (role:0) do bazy danych
   Jako **wejście** przyjmuje JSON w postaci:

```
{
    "username": "string",
    "password": "string",
    "email": "string",
    "phone_number": "string"
}
```

Jako **wyjście** zwraca:
W pyrzpadku sukcesu: "User added", 200
W przypadku braku podania odpowiedniego wejścia:

```
        {"error": "No data provided"}, 400
```

w przypadku powtórzenia nazwy użytkownika bądź jego emaila:

```
        {"error": "User already exists"}, 400
```

2. POST addLibrarian i POST addAdmin przyjmują i zwracają takie same dane jak endpoint addUser 1, jednak w przypadku kiedy chcę ich dodać nieautoryzowany użytkownik (czytelnik, bibliotekarz) zwraca błąd:

```
        {"error": "You are not authorized"}
```

3. POST login - pozwala użytkownikowi zalogować się do systemu. Na wejściu przyjmuje JSON w postaci:

```
    {
        "username": "string",
        "password": "string"
    }
```

Na wyjściu zwraca:
W przypadku powodzenia

```
        {"message": "Login successful"}, 200
```

W przypadku nie poprawnych danych:

```
        {"error": "Username and password are required"}, 400
```

W przypadku nazwy użytkownika nie znajdującej się w bazie:

```
        {"error": "User not found"}, 404
```

W przypadku błędnego hasła:

```
        {"error": "Invalid password"}, 401
```

4. POST addBooks - pozwala na dodanie nowego egzemplarza książki do bazy, jeśli książka wcześniej nie występowała w bazie dodaję ją do bazy.
Na wejściu przyjmuje JSON w postaci:

```
    {
        "title": "string",
        "author": "string",
        "genre": "string"
    }
```

Na wyjściu zwraca:
W przypadku powodzenia:

```
        {"message": "Book added successfully"}, 200
```

W przypadku nieautoryzowanego użytkownika:

```
        {"error": "You are not authorized"}, 400
```

5. GET books - pozwala wyszukać i sprawdzić dostępność książki o zadanym tytule, autorze czy gatunku.
   Na wejście przyjmuje zadany tytuł, autora lub gatunek. Na wyjściu zwraca tablicę książek w postaci JSON:

```
    [
        {
            "title": "string",
            "author": "string",
            "genre": "string",
            "status": "dostepna" | "niedostepna"
        }
    ]
```

6. POST deleteBook - pozwala bibliotekarzowi usunąć egzemplarz książki z bazy (w praktyce ustawić status egzemplarza na 0)
   Na wejściu przyjmuje:

```
{
  "copy_id": "integer"
}
```

   W przypadku powodzenia zwraca:

```
    {"message": "Book deleted successfully"}, 200
```

   W przypadku niepowodzenia zwraca:

```
    {"error": "You are not authorized"}, 400
    {"error": "Book is already deleted"}, 400
```

   1. jeżeli książkę próbuje usunąć nieautoryzowany użytkownik
   2. jeżeli książka jest już usunięta

7. POST reserveBook - pozwala użytkownikowi zarezerwować książki
   Na wejście przyjmuje nazwę użytkownika i tablicę rezerwowanych książek:

```
    {
        "username": "string",
        "books": [
            {
                "title": "string",
                "author": "string"
            }
        ]
    }
```

   W przypadku powodzenia zwraca:

```
    {"message": "Books reserved successfully"}
```

   W przypadku niepowodzenia zwraca:

```
    {"error": "User not found"}, 404
    {'error': 'Some of the selected books are not available for
       reservation, part of reservation made'}, 400
    {"error": "None of the selected books are available for
       reservation"}, 400
```

   1. w przypadku nazwy użytkownika nie znajdującej się w bazie
   2. w przypadku kiedy nie wszystkie książki udało się zarezerwować
   3. w przypadku kiedy nie udało się zarezerwować żadnej książki

8. POST loanBook - pozwala bibliotekarzowi wypożyczyć książkę.
Na wejście przyjmuje:

```
    {
        "user_username": "string",
        "title": "string",
        "author": "string"
    }
```

W przypadku powodzenia zwraca:

```
    {'message': 'Book loaned successfully'}, 200
```

W pyrzpadku niepowodzenia zwraca:

```
    {'error': 'Book not available for loan'}, 404
    {'error': 'Book not found'}, 404
    {'error': 'You are not authorized'}, 400
    {'error': 'User not found'}, 404
    {'error': 'Fine not paid'}, 404
```

1. jeżeli nie ma dostępnej do wypożyczenia książki
2. jeżeli nie znaleziono w bazie podanej książki
3. jeżeli książkę próbował wypożyczyć nieautoryzowany użytkownik
4. jeżeli nie znaleziono użytkownika o podanej nazwie
5. jeżeli użytkownik nie zapłacił kary z poprzedniego wypożyczenia

9. POST returnBook - pozwala bibliotekarzowi przyjąć zwrot książki i nalicza karę jeżeli długość wypożyczenia została przekroczona.
Na wejściu przyjmuje:

```
    {
        "username": "string",
        "title": "string",
        "author": "string"
    }
```

W przypadku sukcesu zwraca:

```
    {"message": "Book returned successfully"},200
```

W przypadku niepowodzenia zwraca:

```
    {"error": "You are not authorized"}, 400
    {"error": "Loan not found"}, 404
```

1. jeżeli zwrot książki próbuje zarejestrować nieautoryzowany użytkownik (inny niż bibliotekarz) 2. jeżeli nie wypożyczenie o podanym wejściu nie zostało znalezione

10. GET getFines - pozwala sprawdzić niezapłacone kary użytkownika.
Na wejście przyjmuje nazwę użytkownika (wymagane)
W przypadku sukcesu zwraca:

```
    {
        "fines": [
            {"fine_id": "integer", "amount": "float", "paid": false}
        ]
    }, 200
```

W przypadku niepowodzenia zwraca:

```
    {"error": "No fines found"}, 404
    {"error": "User not found"}, 404
```

1. jeżeli podany użytkownik nie ma niezapłaconych kar
2. jeżeli nie znaleziono podanego użytkownika

11. POST payFine - pozwala bibliotekarzowi zarejestrować zapłatę kary. Czyli w praktyce zmienić pole paid na True
Na wejście przyjmuje:

```
    {
        "username": "string"
    }
```

W przypadku sukcesu zwraca:

```
    {"message": "Fine paid successfully"}, 200
```

W przypadku niepowodzenia zwraca:

```
    {'error': 'You are not authorized'}, 400
    {'error': 'No fines found'}, 404
```

1. jeżeli zapłacenie kary próbuje zarejestrować nieautoryzowany użytkownik (inny niż bibliotekarz)
2. jeżeli podany użytkownik nie ma kar do zapłacenia

12. GET userLoans - pozwala użytkownikowi sprawdzić wszystkie dotychczasowe wypożyczenia. Na wejście przyjmuje nazwę użytkownika (wymagane). W przypadku powodzenia zwraca:

```
    [
       {
            'username': "string",
            'book_title': "string",
            'author':"string",
            'loan_date': "date",
            'expected_return_date':"date",
            'actual_return_date': "date",
            'fine_amount': "float",
            'fine_paid': "boolean"
        }
    ], 200
```

W przypadku niepowodzenia zwraca:

```
    {'error': 'Username is required'}, 400
    'error': 'User not found'}, 404
```

1. jeżeli nie została podana nazwa użytkownika
2. jeżeli nie znaleziono użytkownika o podanej nazwie

13. GET userReservations - pozwala użytkownikowi sprawdzić wszystkie dotychczasowe rezerwacje. Na wejście przyjmuje nazwę użytkownika(wymagane).
W przypadku sukcesu zwraca:

```
  [
    {
     "username": "string",
     "reservation_date": "date",
     "title": "string",
     "author": "string"
    }
  ]
```

W przypadku niepowodzenia zwraca:

```
    {"error": "Username is required"}, 400
    {'error': 'User not found'}), 404
    {'error': 'No reservations found'}, 404
```

1. jeżeli nie podano nazwy użytkownika 2. jeżeli nie znaleziono użytkownika o poadanej nazwie 3. jeżeli nie znaleziono rezerwacji dla podanego użytkownika

14. POST logout - pozwala wylogować się użytkownikowi, czyszcząc sesje. Nie przyjmuje wejścia. Na wyjściu zwraca:

```
    {'message': 'Logout successful'}, 200
```